



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

RISC-V Data Transfer



RV32I So Far

opname rd rs1 rs2

add rd rs1 rs2

$R[rd] = R[rs1] + R[rs2]$

sub rd rs1 rs2

$R[rd] = R[rs1] - R[rs2]$

Not an array. Verilog syntax
describes operation

$R[rs1]$: data in register rs1



Immediates, Register Zero

- Register Zero, Pseudoinstructions
- Data Transfer Instructions
- Byte-level Data Transfer
- Stack Pointer, Register Names



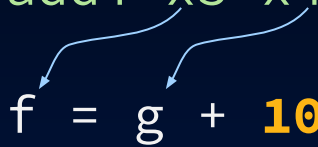
Immediates

- **Immediates** are numerical constants in assembly instructions.
 - Immediate bits are **directly** encoded into instructions
 - So ISA defines separate instructions for immediates

Add Immediate

RISC-V `addi x3 x4 10`

C `f = g + 10;`





Subtract Immediate?

- **Immediates** are numerical constants in assembly instructions.
 - Immediate bits are **directly** encoded into instructions
 - So ISA defines separate instructions for immediates

Add Immediate

RISC-V `addi x3 x4 10`

C `f = g + 10;`

Subtract Immediate?

RISC-V `addi x3 x4 -10`

C `f = g - 10;`

RISC Philosophy: Reduced.

If an operation can be decomposed into **equivalent/simpler operations**, don't include it in the ISA.

Register Zero

- One particular immediate, the number zero (**0**), appears very often in code.
- RISC-V **hardwires** the register zero (**x0**) to value **0**.

```
addi x3 x0 0xff
```

```
      f      g
     x3     x4
```

```
sub  x3 x0 x4
```

```
add  x0 x3 x4
```

What are equivalent C statements, if any?



Yan, SP26



Register Zero

- One particular immediate, the number zero (**0**), appears very often in code.
- RISC-V **hardwires** the register zero (**x0**) to value **0**.

f	g
x3	x4

```
addi x3 x0 0xff           # f = 0xff;
```

```
sub  x3 x0 x4              # f = -g;
```

```
add  x0 x3 x4              # // no-op ("no operation")
```



For convenience, Load Immediate

- **Pseudoinstructions** are convenient instructions in RISC-V.

- One example:

```
addi x3 x0 0xff          # f = 0xff;
```

- Can often be written as **load Immediate** pseudoinstruction

```
li x3 0xff
```

- The assembler then replaces pseudoinstruction with real instruction `addi x3 x0 0xff` to assemble to machine code bits



Announcements

- Register Zero, Pseudoinstructions
- Data Transfer Instructions
- Byte-level Data Transfer
- Stack Pointer, Register Names



Announcements

- Discussion
 - Due to holiday M 2/16, everyone gets attendance points next week
 - That being said, please switch to the new M 1-2 discussion!!! Andrea is great!!!
 - Last day to switch: **today**, but email cs61c@berkeley.edu if you have questions
- Quest:
 - Study by finishing HW 3 early if you can
 - Practice quest out in next 24 hours
 - Sign up for optional orientation this Th or F

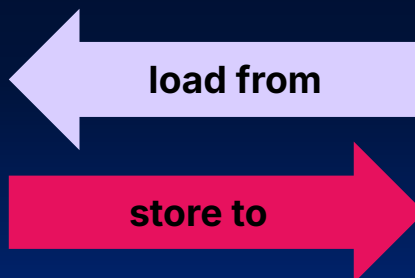
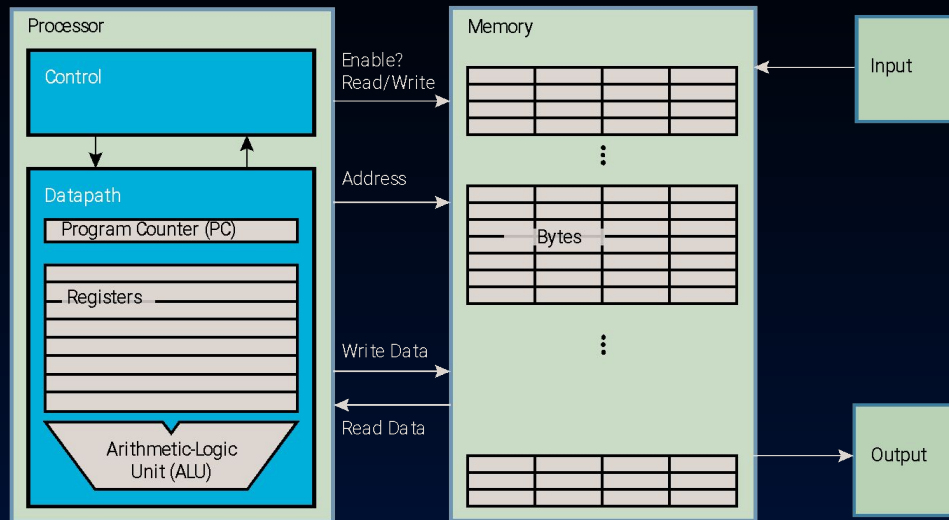


Data Transfer Instructions

- Register Zero, Pseudoinstructions
- Data Transfer Instructions
- Byte-level Data Transfer
- Stack Pointer, Register Names

Load from Memory, Store to Memory

- "Load-Store" Architecture:
 - To perform operations on data, first **load** data from memory into registers.
 - When out of register space, **store** data to memory.





Load from Memory: Load word

load from memory

- Load word syntax:
`lw rd imm(rs1)`
- Means:
 - Compute address: $R[rs1] + imm$
 - (address must be word-aligned)
 - Load 4 bytes from address into $R[rd]$



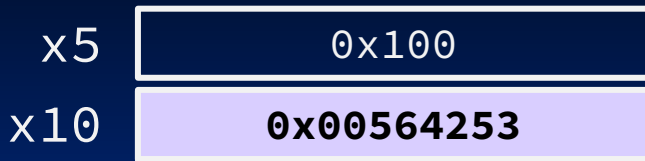
Load Word: Example

load from memory

- Load word syntax:
`lw rd imm(rs1)`
- Means:
 - Compute address: $R[rs1] + imm$
 - (address must be word-aligned)
 - Load 4 bytes from address into $R[rd]$

	+3	+2	+1	+0
...				
0x100	0xB0	0xBA	0xCA	0xFE
0x104	0x44	0x33	0x22	0x11
0x108	0x49	0x52	0x20	0x43
0x10C	0x00	0x56	0x42	0x53
...				

- Example:
`lw x10 12(x5)`



Compute address:
 $0x100 + 12$
 $= 0x10C$



Store to Memory: Store word

store to memory

- Store word syntax:
`sw rs2 imm(rs1)`
- Means:
 - Compute address: $R[rs1] + imm$
 - (address must be word-aligned)
 - Store 4 bytes of $R[rs2]$ to address



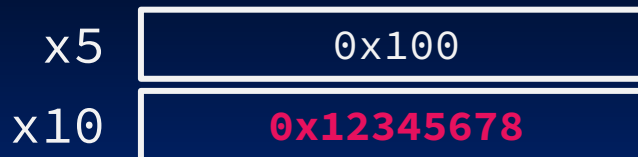
Store word: Example

store to memory

- Store word syntax:
`sw rs2 imm(rs1)`
- Means:
 - Compute address: $R[rs1] + imm$
 - (address must be word-aligned)
 - Store 4 bytes of $R[rs2]$ **to** address

	+3	+2	+1	+0
...				
0x100	0x12	0x34	0x56	0x78
0x104	0x44	0x33	0x22	0x11
0x108	0x49	0x52	0x20	0x43
0x10C	0x00	0x56	0x42	0x53
...				

- Example:
`sw x10 0(x5)`



Compute address:
 $0x100 + 0$
 $= 0x100$



Byte-level Data Transfer

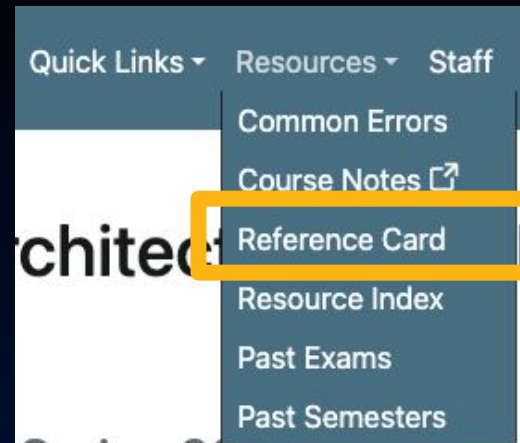
- Register Zero, Pseudoinstructions
- Data Transfer Instructions
- Byte-level Data Transfer
- Stack Pointer, Register Names

RISC-V Green Card Reading Practice (1/2)

- Terminology:
 - Base register:** `rs1`
 - Offset:** `imm` is a constant that must be known at assembly time

Try these conceptual questions out!

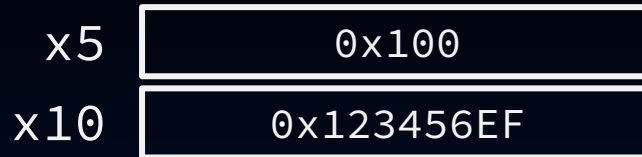
- For `lb` and `sb`, what does the verilog **description** mean?
- Why might we need **two instructions** for “load byte” (`lb`, `lbu`)? Why **only one** for `sb`?



Yan, SP26



RISC-V Green Card Reading Practice (2/2)



- Execute these instructions:

```
sb x10 0(x5)
lb x10 0(x5)
```

	+3	+2	+1	+0
...				
0x100	0xB0	0xBA	0xCA	0xFE
0x104	0x44	0x33	0x22	0x11
0x108	0x49	0x52	0x20	0x43
0x10C	0x00	0x56	0x42	0x53
...				

- For `lb` and `sb`, what does the verilog **description** mean?
- Why might we need **two instructions** for “load byte” (`lb`, `lbu`)? Why **only one** for `sb`?



Yan, SP26

HLL vs. Assembly: Data representation

C, Java

- In high-level languages, **variable types determine operation**.

```
uint8_t x, *y;
```

```
...
```

```
x = *y;
```

```
x = 4 * x;
```

RISC-V

- Operation** determines "type," i.e., how register contents are treated.
 - as value, memory address, etc.

```
lb x4 0(x3)
```

```
add x2 x3 x4
```

RV32I registers **have no type**.
Use RV instruction **opnames** to infer the
analogous C code.

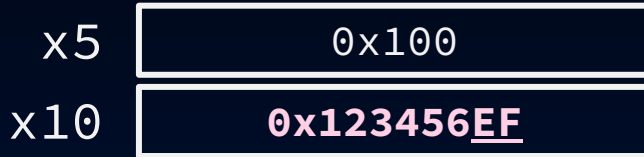


Store Byte

store to memory

- For `sb`, we store the **least significant byte**.
- Example:

`sb x10 0(x5)`



	+3	+2	+1	+0
...				
0x100	0xB0	0xBA	0xCA	0xEF
0x104	0x44	0x33	0x22	0x11
0x108	0x49	0x52	0x20	0x43
0x10C	0x00	0x56	0x42	0x53
...				

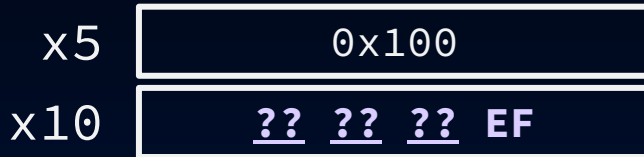


Load Byte...?

load from memory

- For `lb`, we load one byte into a register.
- Example:

`lb x10 0(x5)`



	+3	+2	+1	+0
...				
0x100	0xB0	0xBA	0xCA	0xEF
0x104	0x44	0x33	0x22	0x11
0x108	0x49	0x52	0x20	0x43
0x10C	0x00	0x56	0x42	0x53
...				

We must put an 8-bit integer into a 32-bit word. How?

1b: Sign-extend, lbu: zero-extend

- Load byte depends on integer representation.
 - 2 instructions for loading in 8-bit integers!

x10 ?? ?? ?? EF

lb x10 0(x5)

Load a signed integer.

Sign-extend to fill register word.

↓
 0b1...111011111
 0xFFFFFFEF

0xEF = -17

lbu x10 0(x5)

Load an unsigned integer.

Zero-extend to fill register word.

0b0...011011111
 0x000000EF

0xEF = 239



Stack Pointer, Register Names

- Register Zero, Pseudoinstructions
- Data Transfer Instructions
- Byte-level Data Transfer
- Stack Pointer, Register Names



Register Conventions

- RISC-V **register convention** gives common **register names** to each register. This convention helps cleaner, faster assembly instructions.
 - Temporary registers t0, t1, t2
 - Stack pointer (yes, that one) sp: start of a usable block of memory on the stack
- If we have too many data words for registers, we must use memory.
 - Let's try an example!

#	Name	Description
x0	zero	Constant 0
x1	ra	<i>Return Address</i>
x2	sp	Stack Pointer
x3	gp	Global Pointer
x4	tp	Thread Pointer
x5	t0	<i>Temporary Registers</i>
x6	t1	
x7	t2	

See [green card!](#)

Example (1/10): Stack Frame

```
int a = 5;  
char b[] = "string"; // Array will get stored on stack  
int c[10];  
uint8_t d = b[3];  
c[4] = a+d;  
c[a] = 20;
```

a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)

First, assign each variable to some offset from our stack pointer sp.

- Analogous to local variables in stack frame
- Exact addresses don't matter, so long as we're consistent



First two lines. Try it yourself!

a: 0(sp) c: 12(sp)
b: 4(sp) d: 52(sp)

```
int a = 5;  
char b[] = "string";  
int c[10];  
uint8_t d = b[3];  
c[4] = a+d;  
c[a] = 20;
```

```
1 li t0 5          # R[t0] = 5  
2 sw t0 0(sp)      # store int a on stack  
3 li t0 0x69727473 # ASCII stri  
4 sw t0 4(sp)      # little endian  
5 li t1 0x0000676E # ASCII ng\0\0  
6 sw t1 8(sp)
```



	+3	+2	+1	+0
0(sp)	00	00	00	05
4(sp)	69	72	74	73
8(sp)	00	00	67	6E
12(sp)	FA	CE	FA	CE
16(sp)	FA	CE	FA	CE



Line 1: Initialize integer

a: 0(sp) c: 12(sp)
b: 4(sp) d: 52(sp)

```
➡ int a = 5;  
  char b[] = "string";  
  int c[10];  
  uint8_t d = b[3];  
  c[4] = a+d;  
  c[a] = 20;
```

t0 0x0000 0005

```
1  li t0 5            # R[t0] = 5  
2  sw t0 0(sp)       # store int a on stack
```

	+3	+2	+1	+0
0(sp)	00	00	00	05
4(sp)	FA	CE	FA	CE
8(sp)	FA	CE	FA	CE
12(sp)	FA	CE	FA	CE
16(sp)	FA	CE	FA	CE



Line 2: Initialize string?

a: 0(sp) c: 12(sp)
b: 4(sp) d: 52(sp)

```
int a = 5;  
char b[] = "string";  
int c[10];  
uint8_t d = b[3];  
c[4] = a+d;  
c[a] = 20;
```

```
3 li t0 0x73  
4 sb t0 4(sp)  
5 li t0 0x74  
6 sb t0 5(sp)  
7 li t0 0x72  
8 sb t0 6(sp)
```

```
9 li t0 0x69  
10 sb t0 7(sp)  
11 li t0 0x6E  
12 sb t0 8(sp)  
13 li t0 0x67  
14 sb t0 9(sp)  
15 sb x0 10(sp)
```

ASCII (0x)	's'	't'	'r'	'i'	'n'	'g'	'\0'
	73	74	72	69	6E	67	00

- Seems costly!
- Can we do better?

	+3	+2	+1	+0
0(sp)	00	00	00	05
4(sp)	69	72	74	73
8(sp)	FA	00	67	6E
12(sp)	FA	CE	FA	CE
16(sp)	FA	CE	FA	CE

Gray: (random garbage)





Read course notes for
the rest of the example.
See you next time...!

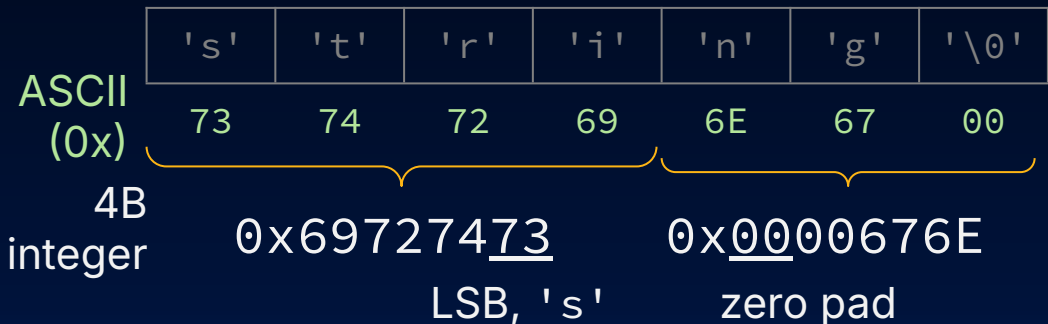


Line 2: String, better

a: 0(sp) c: 12(sp)
b: 4(sp) d: 52(sp)

```
int a = 5;  
char b[] = "string";  
int c[10];  
uint8_t d = b[3];  
c[4] = a+d;  
c[a] = 20;
```

```
3 li t0 0x69727473 # ASCII string  
4 sw t0 4(sp) # little endian  
5 li t1 0x0000676E # ASCII ng\0\0  
6 sw t1 8(sp)
```



Assume little endian

	+3	+2	+1	+0
0(sp)	00	00	00	05
4(sp)	69	72	74	73
8(sp)	00	00	67	6E
12(sp)	FA	CE	FA	CE
16(sp)	FA	CE	FA	CE

Gray: (random garbage)

Yan, SP26





Line 3: Declare array

a: 0(sp) c: 12(sp)
b: 4(sp) d: 52(sp)

```
int a = 5;  
char b[] = "string";  
➡ int c[10];  
uint8_t d = b[3];  
c[4] = a+d;  
c[a] = 20;
```

- No data moves between registers and memory!
- **No instructions needed**—leave things as-is!

	+3	+2	+1	+0
0(sp)	00	00	00	05
4(sp)	69	72	74	73
8(sp)	00	00	67	6E
12(sp)	FA	CE	FA	CE
16(sp)	FA	CE	FA	CE

Yan, SP26



Line 4: Load/store byte

a: 0(sp) c: 12(sp)
b: 4(sp) d: 52(sp)

```
int a = 5;  
char b[] = "string";  
int c[10]  
uint8_t d = b[3];  
c[4] = a+d;  
c[a] = 20;
```

t0

0x0000 0069

```
7 lb t0 7(sp) # 4(sp) from b, 3(sp) from [3]  
8 sb t0 52(sp) # store into d
```

	+3	+2	+1	+0
0(sp)	00	00	00	05
4(sp)	<u>69</u>	72	74	73
8(sp)	00	00	67	6E
...	...			
52(sp)	FA	CE	FA	69

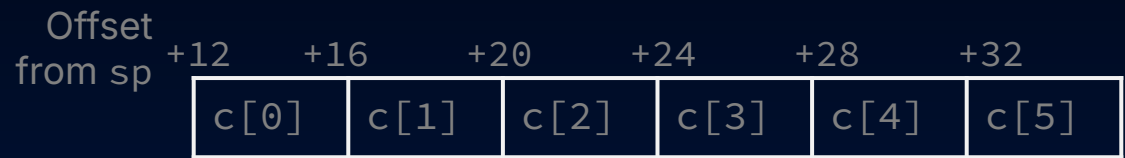
Yan, SP26



Line 5: Constant array index

a: 0(sp) c: 12(sp)
b: 4(sp) d: 52(sp)

```
int a = 5;  
char b[] = "string";  
int c[10];  
uint8_t d = b[3];  
➡ c[4] = a+d;  
c[a] = 20;
```



- Pointer arithmetic: 4*sizeof(int) = 16
- 16 bytes after 12(sp) → 28(sp)

```
9  lw t0 0(sp)      # load a  
10 lbu t1 52(sp)    # load d  
11 add t2 t0 t1      # R[t2] = a+d  
12 sw t2 28(sp)      # 12(sp) from c, 16(sp) from [4]
```

t0	0x0000 0005
t1	0x0000 0069
t2	0x0000 006E

	+3	+2	+1	+0
0(sp)	00	00	00	05
4(sp)	69	72	74	73
...	...			
28(sp)	00	00	00	6E
...	...			
52(sp)	FA	CE	FA	69

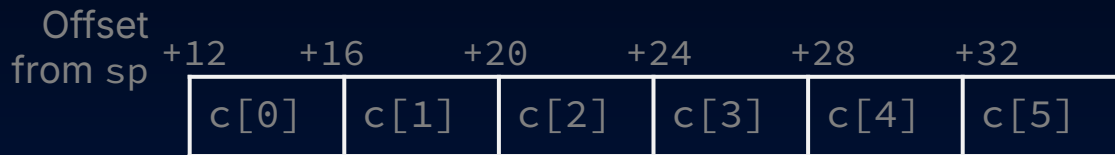




Line 6: Variable array index

a: 0(sp) c: 12(sp)
b: 4(sp) d: 52(sp)

```
int a = 5;  
char b[] = "string";  
int c[10];  
uint8_t d = b[3];  
c[4] = a+d;  
➡ c[a] = 20;
```



t0	0x0000 0014
t1	0x0000 0005 L14
	0x0000 0014 L15
	0x0000 0020 L16
	(sp+32) L17

```
13 li t0 20          # R[t0] = 20  
14 lw t1 0(sp)       # R[t1] = a  
15 slli t1 t1 2       # 5*sizeof(int) = 5*4 = 5<<2  
16 addi t1 t1 12      # t1 from [a], 12 from c  
17 add t1 t1 sp       # compute &c[a]  
18 sw t0 0(t1)       # c[a] = 20
```

Yan, SP26

Example: Full Code

```
1 int a = 5;
2 char b[] = "string";
3 int c[10];
4 uint8_t d = b[3];
5 c[4] = a+d;
6 c[a] = 20;
```

a: 0(sp)
b: 4(sp)
c: 12(sp)
d: 52(sp)

```
1 li t0 5 # R[t0] = 5
2 sw t0 0(sp) # store int a on stack
3 li t0 0x69727473 # ASCII string
4 sw t0 4(sp) # store first part of string
5 li t1 0x0000676E # ASCII ng\0\0
6 sw t1 8(sp) # store rest of string
7 lb t0 7(sp) # 4(sp) from b, 3(sp) from [3]
8 sb t0 52(sp) # store into d
9 lw t0 0(sp) # load a
10 lbu t1 52(sp) # load d
11 add t2 t0 t1 # R[t2] = a+d
12 sw t2 28(sp) # 12(sp) from c, 16(sp) from [4]
13 li t0 20 # R[t0] = 20
14 lw t1 0(sp) # R[t1] = a
15 slli t1 t1 2 # 5*sizeof(int) = 5*4 = 5<<2
16 addi t1 t1 12 # t1 from [a], 12 from c
17 add t1 t1 sp # compute &c[a]
18 sw t0 0(t1) # c[a] = 20
```